

ADR for a Local-First Offline Personal Finance Planner

Executive summary

I propose building a **single-user desktop app** that uses an **encrypted local SQLite database (via SQLCipher)**, a **JavaScript/TypeScript or Python frontend**, and an import pipeline for CSV/OFX/QFX/QIF data. In practice, this means using **Electron with Node.js (JS/TS)** or **Python (Tkinter/PySide)** or **.NET (Avalonia)** for cross-platform UI, since those align with the user's existing skills and require minimal setup. The DB will be a local SQLite file with AES-256 encryption (SQLCipher), so all data at rest is encrypted. We'll encrypt the database key using a user passphrase (Argon2id KDF per OWASP) so the key can be changed without re-encrypting data ¹ ².

The import strategy focuses on **structured file imports**. Supported formats include **CSV** (with per-bank column templates), **OFX/QFX** (standard XML formats), and **QIF** (legacy Quicken), since major institutions (Chase, Wells Fargo, Wealthfront, etc.) document these exports ³ ⁴ ⁵ ⁶. PDF statement imports are a fallback for banks without exports, parsed by text and template rules (no OCR in MVP). Imported transactions are normalized into a canonical schema (date, amount, category, etc.). We will dedupe by bank-provided ID or by a fingerprint (date+amount+payee) and allow manual review of fuzzy matches.

The app will have a **deterministic projection engine** (cash-flow, net-worth, debt amortization, simple returns) and defer Monte Carlo simulations until later. All data lives in the local database; user files (statements) are only temporarily staged and then optionally archived encrypted. Backups are explicit exports (encrypted DB copy + metadata) and CSV/JSON exports of raw data. The app runs entirely offline with no cloud sync.

In summary, I recommend:

- **DB:** SQLite+SQLCipher (file-based, multi-platform, encrypted) ⁷.
- **App shell:** Electron (JS/TS) or Python desktop (Tkinter/PySide) or .NET Avalonia (C#) – all are cross-platform with installer support. Electron likely makes packaging easiest for a JS-fluent dev ⁸ ⁹ ¹⁰.
- **Key mgmt:** Passphrase → Argon2id → KEK encrypting a random DEK (SQLCipher key) ².
- **Imports:** CSV/OFX/QFX/QIF first (supported by major vendors) ³ ⁴; PDF templates fallback.
- **Schema:** Canonical transaction table with fields like date, amount, payee, category, type, etc.
- **Projections:** Monthly forecast with fixed assumptions (no randomness).
- **Security:** Use OS file permissions (0700), SQLCipher encryption, `temp_store=MEMORY`, and no storage of sensitive plaintext beyond what's absolutely needed ¹.

This ADR outlines the architecture for an MVP with these choices and notes alternatives (in tables), a migration plan, and testing criteria.

Context & problem statement

We need a **local-first financial planner** for a middle-class working person, tracking incomes, spending, net worth, goals, and asset allocation. It must run on Windows/macOS/Linux without requiring cloud services or internet. The user is comfortable with **JavaScript/TypeScript, Python, Java, or C#**; we should avoid forcing a new language like Rust.

The app must persist data between sessions and handle sensitive financial data safely. Based on the PRD, it will import data from multiple accounts (e.g. Robinhood, Chase cards, Wells Fargo checking, Vanguard 401k, student loan servicer, Wealthfront, etc.). Most of these institutions provide **downloadable transactions or statements in CSV, OFX/QFX, QIF, or PDF** formats. For example, Chase's online "Spending Planner" supports CSV, QFX, QIF, and QBO formats on desktop ³. Wells Fargo lets users download CSV (ASCII) or Quicken Web Connect (QFX) files ⁴. Wealthfront lets users export a QFX (Quicken) file ⁵. Edfinancial (student loans) provides export to spreadsheet ⁶. (Vanguard offers CSV for personal accounts). In short, **structured exports exist for most data sources** and will be our import pipeline's primary input.

We assume no bank logins or API aggregation. Data is imported by the user manually (e.g. dragging exported files into the app). All processing and storage is offline.

Because the data is sensitive, we will encrypt it on disk. The offline threat model we consider includes laptop theft or backup compromise. We will not have cloud threats. OWASP's cryptographic storage guidelines apply: encrypt data with strong keys, use OS-provided key storage when possible, and separate keys from encrypted data ² ¹.

Given the user's technology background and minimal setup requirement, we will favor stacks they know (Electron/Node, Python, or .NET) and document how each works. The ADR below records the chosen architecture and its justification, plus alternatives.

Architecture decisions and rationale

- **Storage:** We choose **SQLite with SQLCipher**. SQLite is ideal for desktop, file-based apps and requires no administration ⁷. SQLCipher transparently encrypts the entire DB (AES-256), including its rollback journal and WAL, so no residual plaintext is written to disk. This matches OWASP advice to encrypt at rest. We set `PRAGMA temp_store=MEMORY` to minimize sensitive data in temporary files and run periodic checkpoints.
- **Key management:** We generate a random 256-bit data key (DEK) for SQLCipher and protect it with a passphrase-derived KEK. Concretely, the app derives an Argon2id key from the user's passphrase and uses it to encrypt the DEK in a local metadata file (or OS keystore). This follows OWASP guidance: "generate a KEK from user input to encrypt a randomly generated DEK," allowing passphrase changes without re-encrypting the entire database ². We do not hardcode or reuse the database password. If the user forgets their passphrase, the DEK is lost and the DB cannot be opened.
- **Application shell:** Because I know JavaScript/TypeScript and want a smooth developer experience, I will use **Electron** (Chromium + Node). Electron apps natively run on Windows, macOS, and Linux ⁸ and can be packaged into installers (DMG/MSI/AppImage) via community tools. If I were a Python

dev instead, I could alternatively use **PyInstaller** to bundle a Python/Tkinter or PySide app; PyInstaller works on Windows, macOS, Linux (built on each) ⁹ . If C# is preferred, I could use **.NET 7/8 with Avalonia UI**, which yields a single codebase for Windows, Mac, Linux ¹⁰ . Electron likely minimizes custom config and leverages many frontend libraries. A Node/Express backend or IPC to SQLite is simple in Electron.

- **Import pipeline:** We will focus on **file-based imports**. Supported formats: CSV (bank-specific format), OFX/QFX (Intuit's open format), QIF, and PDF-statement parsing. This matches real user data: Chase CSV/QFX/QIF ³ , Wells Fargo CSV/QFX ⁴ , Wealthfront QFX ⁵ , etc. We should use mature parser libraries (e.g. `csv-parse` , `ofxtools` in Python, or similar NPM packages) and institution-specific templates (column mapping, date format, currency). The code will tag each import with source metadata and raw JSON for auditing. The app will prevent duplicates by checking `FITID` or transaction ID first, then a fingerprint (date+abs(amount)+payee) if no ID is present. It will merge pending vs posted transactions intelligently (same amount and merchant, few days apart). PDF import is a fallback: for statements where no export exists, we'll extract text by pre-existing templates (e.g. tabular PDF) without OCR, flag fields with low confidence, and allow the user to review.
- **Data model:** Transactions will be normalized into a ledger-style table. Key fields include: `txn_id` (UUID) , `account_id` , `date` , `amount` , `currency` , `type` (enum: expense, income, transfer, buy, sell, dividend, fee, interest, etc.) , `payee` , `category` , `memo` , `external_id` (bank-supplied FITID or check #) , `status` (pending/posted), and optional `security_symbol` , `quantity` , `price` for investment trades. We'll also maintain account balances, positions (if import provides cost basis or quantities), savings goals, and asset allocation targets. This schema supports the needed dashboards (net worth over time, cash flow, debt schedule, portfolio breakdown). All data is stored in the encrypted SQLite file.
- **Projection engine:** The MVP engine will be **deterministic**. It will take current balances, recurring incomes/expenses, planned contributions, debt payoff schedules, and simple rate-of-return assumptions to project cash and net worth monthly. Monte Carlo simulation is deferred to a later phase once the core model works.
- **File layout and backups:** All app data resides in a local application-data folder (e.g. `%LOCALAPPDATA%/MyPlanner` on Windows, `~/Library/Application Support/MyPlanner` on Mac, `~/.myplanner` on Linux). This folder contains `planner.db` (encrypted SQLite) plus optional sidecars `planner.db-wal` / `-shm` if WAL mode is used, and a small metadata file storing the encrypted DEK and KDF salt. Imported source files go to a temp subfolder during import then are deleted (unless the user opts to archive them encrypted). The app will include an "Export Backup" command that writes out an encrypted copy of the DB (e.g. via `sqlcipher_export()` or `VACUUM INTO encrypted_backup.db`) and the current key metadata, all encrypted with the same passphrase. We'll also allow one-click CSV/JSON export of transactions for portability.
- **Threat model & security:** We assume an attacker may gain access to the user's disk image or steal a laptop. SQLCipher encryption protects data at rest; as OWASP advises, we rely on the OS for user-only file permissions (0700/600) ¹ . We avoid writing raw transactions to temporary files (use `temp_store=MEMORY`). If WAL journaling is enabled for concurrency, it remains encrypted by SQLCipher; we will checkpoint/truncate the WAL on clean shutdown. We avoid any internet

connectivity, third-party services, or embedded credentials that could leak. By design, the app never sends data out or requires login keys for banks.

- **Packaging & deployment:** For Electron, we use `electron-builder` or `electron-forge` to create native installers (.exe/.msi for Win, .dmg for Mac, .AppImage/.deb/.rpm for Linux). This is an all-JS toolchain and a single GitHub repo can contain the `src/` for UI and `package.json`. For Python, we'd rely on PyInstaller to create OS-specific executables (requiring separate build jobs per OS) ⁹. For C#, a self-contained build can be packaged with WiX or zip. In all cases, CI can build all targets automatically.
- **Migration & rollback plan:** Development will use SQLite schema migrations. On first app run, create `planner.db` (new SQLCipher file) and prompt for passphrase. We record a `schema_version` in a migrations table. On upgrades, the app will perform a “preflight” backup: copy the encrypted DB file to `backups/` before altering. Migration SQL (Add column or table) runs in a transaction. If any migration fails, the app restores from backup and warns the user. For SQLCipher library upgrades or encryption parameter changes, use `cipher_migrate` or `sqlcipher_export()` as per [SQLCipher docs] ¹⁰ (the upgrade path will be documented carefully). User-visible rollback is simply restoring from an earlier backup.

Alternatives considered

Database

Option	Pros	Cons	Decision	Sources
SQLCipher + SQLite	File-based (no server), ACID, well-tested for desktop apps ⁷ ; cross-platform; encryption of entire DB & journals; no separate service needed.	Slightly more complex dev setup (need SQLCipher build), though libraries exist for JS/Python/.NET.	Chosen	⁷ ¹
Plain SQLite	Very simple; no extra encryption layer; same file DB benefits.	No at-rest encryption (only file-system); relies on full-disk encryption; exports/backups unencrypted.	Rejected	¹ (encryption needed)
DuckDB	Single-file OLAP DB; good for analytic queries; built-in concurrency control for in-process use ¹¹ ; performant for bulk queries.	Not designed for many small write transactions ¹² ; only one writer process allowed ¹³ ; niche libs in JS/Python.	Rejected for MVP	¹¹ ¹³

Option	Pros	Cons	Decision	Sources
Local PostgreSQL	Full-featured SQL engine; multi-user; strong ACID; remote future-proofing.	Must run a background server/service; heavy install on user machine; overkill for single user; more setup complexity.	Rejected	Common knowledge, e.g. [36†L0]

Application shell

Option	Pros	Cons	Decision	Sources
Electron (JS/TS)	JS/TS frontend and Node backend; cross-platform; stable web engine ⁸ ; rich ecosystem; easy packaging via electron-builder; user's skillset.	Bundles Chromium ~100MB; requires Node dev environment; app size larger.	Chosen	⁸
Electron + Node.js	(As above; explicitly includes ability to use Node modules and SQLite packages)	Same as Electron; native modules for SQLCipher exist (e.g. [rusqlite for Node]) needs build.	Same as above	–
Python desktop	Mature GUI libs (Tkinter, PySide, PyQt); single exe via PyInstaller; cross-platform ⁹ ; uses user-known language.	UI look-and-feel is basic (Tk) or needs PySide UI work; PyInstaller is not cross-compiler ⁹ ; Python path and packaging slightly tricky; performance.	Possible (if dev prefers Python)	⁹
.NET / Avalonia (C#)	Cross-platform .NET UI (Avalonia) ¹⁰ ; one codebase for Win/Linux/Mac; Windows dev familiarity; .NET has encryption APIs; strong tooling.	Requires .NET runtime or self-contained ~50MB; learning curve if not known; packaging via MSI/zip.	Possible (if dev prefers C#)	¹⁰
Java GUI	Write once run anywhere; desktop libs (Swing/JavaFX); strong typing.	Requires JRE on user's machine (or bundled ~50MB); not as common for desktop tools today.	Rejected	(General knowledge)
Native CLI (Node/Python)	Very simple to implement; minimal packaging.	Very poor UX for finance (no GUI); doesn't fit "planner" concept.	Rejected	–

Import strategy

Option	Pros	Cons	Decision	Sources
Structured files (CSV/ OFX/QFX/QIF)	Aligns with how banks provide exports; deterministic parsing; works offline; easy to test ³ ⁴ .	Need mapping logic per institution; must update templates as sites change.	Chosen	³ ⁴
OFX/QFX API (FDX)	Future-proof standard; automation possible; less manual export.	Requires API connectivity, OAuth, and partner membership (FDX/API keys) ¹⁴ ; out of scope for offline MVP.	Deferred	¹⁴
PDF/OCR-first	Works even if no structured data; uses official statements.	High error rate; heavy engineering; outside MVP scope.	Only fallback	³ (shows PDF mobile)
Manual entry only	Simple to implement; no imports.	Impossible for real data needs; user overload.	Rejected	(Context)

Key storage and encryption

Option	Pros	Cons	Decision	Sources
Passphrase-KDF + encrypted key (DEK)	Follows OWASP: derive KEK with Argon2id from user passphrase, encrypt random DEK ² ; allows passphrase changes without rewriting DB; strong security.	Slightly more code (must manage two secrets); need to secure metadata file.	Chosen	² ¹
OS keystore / ProtectedData	Very secure storage tied to OS user; user doesn't have to type passphrase every time.	UX complexity (unlock prompt, keychain differences on Linux); 3rd-party integration overhead; cross-platform inconsistency.	Alternative (optional)	¹
SQLCipher passphrase only	Simpler: user passphrase directly opens DB via SQLCipher's internal KDF.	Passphrase change requires full PRAGMA rekey ; less key separation; weaker per OWASP ² (can't re-wrap easily).	Rejected for core	²

Option	Pros	Cons	Decision	Sources
No encryption	Fast and easy.	Completely unacceptable for sensitive data.	Rejected	-

Migration and rollback plan

Timeline (Mermaid diagram): Development will proceed in stages from zero to MVP, then to enhancements.

```

timeline
    title Development timeline to MVP and beyond
    2026-05-01 : Project kickoff - repo scaffolding, choose stack (Electron/Node or Python/.NET), CI setup.
    2026-06 : Secure storage prototype - integrate SQLCipher, implement vault+Argon2 passphrase unlock.
    2026-07 : Schema & API - design DB schema, build repository layer for transactions/accounts.
    2026-08 : CSV import - implement CSV adapters for Chase, Wells Fargo, Vanguard, Edfinancial.
    2026-09 : OFX/QFX/QIF import - add OFX parsing (banks, brokers), QIF parsing, dedupe logic.
    2026-10 : Basic UI - transaction list, net worth and spending charts, account setup UI.
    2026-11 : Projections - add cash-flow/goal projections engine, simple forecasting.
    2026-12 : Backup & packaging - implement encrypted backup export; build installers for Windows/macOS/Linux.
    2027-01 : PDF import fallback - parse remaining statement formats into transactions.
    2027-02 : Testing & polish - comprehensive QA, documentation, and respond to user feedback.
    2027-03+ : Future - Monte Carlo simulator, OS keychain integration, live price import, FDX API if ever needed.

```

Migration strategy: We'll version database schemas using a migration table. Before any upgrade, the app will make an encrypted backup of the current DB (journals included). Then run ALTER TABLE or other SQL migration in a transaction. If it fails, we stop and restore the backup. For SQLCipher library changes, we'll call `cipher_migrate` on the first open to update the file format if needed. We store a `user_version` in SQLite to detect whether migration is needed on startup. All migrations will be scripted so that rollbacks (back to an earlier version) are just a matter of restoring a backup file.

Rollback plan: If a new release proves faulty, users can restore their previous encrypted backup. We should retain several dated backups. The application will warn on startup if it finds an empty or corrupted DB (e.g. bad key). In that case we instruct the user to restore a backup or re-enter data. Schema changes will be documented, but a failure during migration triggers an automatic revert to last-known-good snapshot

taken just before applying the change. This mirrors OWASP's advice to always have key material ready to rotate or recover ² .

Security and operational notes

- **Encryption:** SQLCipher (AES-256) protects all data. We never store the plaintext key; the DEK is encrypted with Argon2id (configurable work factor) using the user's passphrase. OWASP's Cryptographic Storage guidelines recommend exactly this DEK/KEK separation ² . We will log out (clear keys) after a period or on app suspend.
- **Access control:** The app directory and files will be created with restrictive permissions (user only). On Windows, we use `%LOCALAPPDATA%\MyApp` (ACLs by default restrict to the user). On Linux/Mac, we set `chmod 700` for folders and `600` for files. We rely on the OS for file access security ¹ .
- **WAL handling:** We will use WAL journaling for its performance and concurrency benefits. SQLCipher encrypts the WAL pages, so this is safe ¹⁴ . We will enforce `PRAGMA synchronous=FULL` to avoid data loss on crash. On clean shutdown or backup, we run `PRAGMA wal_checkpoint(TRUNCATE)` to minimize leftover WAL files. We may also set `PRAGMA journal_size_limit` to prevent unbounded growth of rollback files.
- **Temporary files:** We set `temp_store = MEMORY` on each SQLite connection to avoid disk temp files. Any intermediate files (like CSV import) are deleted promptly.
- **Dependencies:** We trust libraries (sqlite + SQLCipher, CSV/ofx parsers) but will pin versions and audit them. No network requests will be made, and no secret credentials are stored other than the user's passphrase (never logged).
- **Backup/Export:** The user's **backup command** will produce an encrypted bundle including the DB file and metadata (salt, encrypted DEK). For portability, we can export CSV/JSON as a convenience (e.g. `File → Export Transactions`). The export formats are plaintext (CSV), assuming the user knows how sensitive it is (e.g. won't share CSV on internet).
- **Packaging:** We will leverage official tooling. Electron apps can be packaged on a build server into MSI, DMG, AppImage, etc ⁸ . For Python, PyInstaller creates OS-specific executables (must run on each OS) ⁹ . For .NET/Avalonia, `dotnet publish` can produce a self-contained app. CI pipelines will automate building each target.
- **Updates:** If using Electron, we can wire in auto-update modules. The ADR assumes users get major upgrades by replacing the app (the encrypted DB is retained on upgrade, with migrations handling any schema change).

Testing and acceptance criteria

We will verify the architecture with these tests and checks:

Category	Test / Check	Acceptance criteria
Storage	Initialize new DB (enter passphrase), close, reopen	Data persists; wrong passphrase yields error (cannot open DB)
Encryption	Inspect on-disk files (strings) after import	No human-readable account names or amounts appear in DB file or WAL
Imports	Import sample files from Chase, Wells Fargo, Wealthfront, Edfinancial, Vanguard (CSV/QFX)	Transactions load correctly; no duplicates on re-import; categories parsed
Parsing	Import sample QIF and PDF (for one example vendor)	Structured data extracted; transactions complete; any missing fields flagged
Deduplication	Import overlapping date ranges twice	Only unique transactions remain; duplicates identified and merged appropriately
Transactions	Enter a test transfer between two accounts	Two linked transactions created; net worth unchanged
Balances	Import statement with running balance, then manual adjust	The balance field in app matches statement data
Projections	Set up recurring income/expense, forecast 12 months	Forecast values match manual calculations within 1%; scenarios stable on rerun
Backups	Create backup export and restore into fresh app	Restored data matches original; DB opens with same passphrase
Migration	Simulate adding a new column (migration)	Migration runs without error; <code>user_version</code> updated; old backups still openable
Permissions	Inspect app-data folder on each OS	Folders/files are user-read/write only; no global permissions set
Concurrency	Run import and UI queries simultaneously	No deadlocks or corruption; readers see consistent view (WAL expected)
Packaging	Install on Windows/Mac/Linux	App launches; can set up accounts; perform import and backup without error
Overall	Manual code review of secrets handling	No hard-coded keys; passphrase not logged or leaked

Each test should be automated where possible (unit tests for import/parsing, integration tests for DB migrations) and manually verified for packaging and UI. Security checks include auditing that OWASP “secure storage” practices are followed ¹.

Open questions and assumptions

- **Languages:** I assume one of JS/TypeScript (Electron), Python, or C# will be chosen. This ADR is written language-agnostic. For example, the SQLite/SQLCipher usage would be through `better-sqlite3` or `sqlite3` (with a SQLCipher build) in Node; or via Python's `pysqlcipher3`; or .NET's `Microsoft.Data.Sqlite` with encryption. (If the team later embraces Rust, we could issue a Rust-specific ADR, but that's optional.)
- **OS targets:** We assume Windows, macOS, Linux. Mobile (iOS/Android) is out of scope.
- **Institution support:** Public docs confirm exports for key accounts. Sequence and TruStone exports were listed but not verified publicly. We assume they have some export (else data can be manually added). If not, they may enter data via CSV or PDF in future.
- **Prices and valuations:** We assume no live market prices. Portfolio values come from statements or user entries. Price lookup is a later feature.
- **Financial logic:** This ADR does not cover accounting double-entry intricacies; it assumes transactions are sufficient without modeling equity accounts in detail. If needed, we could later add a separate ledger layer.
- **Security scope:** We assume local device malware or keylogger attacks are out of scope; OS-level disk encryption (FileVault, BitLocker) is recommended but separate.
- **Trust in libs:** We rely on well-known crypto (SQLCipher) and parser libraries, but no guaranty is implied; we'll track CVEs for them.

This ADR provides a clear blueprint: use a local encrypted SQLite database, a JS/Python/C# GUI, structured imports, and OWASP-aligned key management. All key decisions are justified by user constraints, technical fit, and security guidelines.

1 2 Cryptographic Storage - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

3 What's Chase Spending Planner? | Chase

<https://www.chase.com/personal/banking/education/budgeting-saving/what-is-chase-spending-planner>

4 Account Activity Questions

<https://www.wellsfargo.com/help/online-banking/activity-faqs/>

5 Can I export my Wealthfront account data to another personal finance tool like Quicken? – Wealthfront Support

<https://support.wealthfront.com/hc/en-us/articles/209353346-Can-I-export-my-Wealthfront-account-data-to-another-personal-finance-tool-like-Quicken>

6 Frequently Asked Questions – Edfinancial Services

<https://edfinancial.studentaid.gov/faqs>

7 Appropriate Uses For SQLite

<https://sqlite.org/whentouse.html>

8 Build cross-platform desktop apps with JavaScript, HTML, and CSS | Electron

<https://www.electronjs.org/>

9 PyInstaller Manual — PyInstaller 6.20.0 documentation

<https://pyinstaller.org/en/stable/>

10 Avalonia UI: The Cross-Platform .NET UI Framework for Desktop, Mobile and Web

<https://avaloniaui.net/>

11 12 13 Concurrency – DuckDB

<https://duckdb.org/docs/current/connect/concurrency>

14 OFX Work Group - Financial Data Exchange

<https://financialdataexchange.org/about-fdx/ofx-work-group/>